

H1 Arquitectura Base y Seguridad: Multi-Tenancy y RBAC

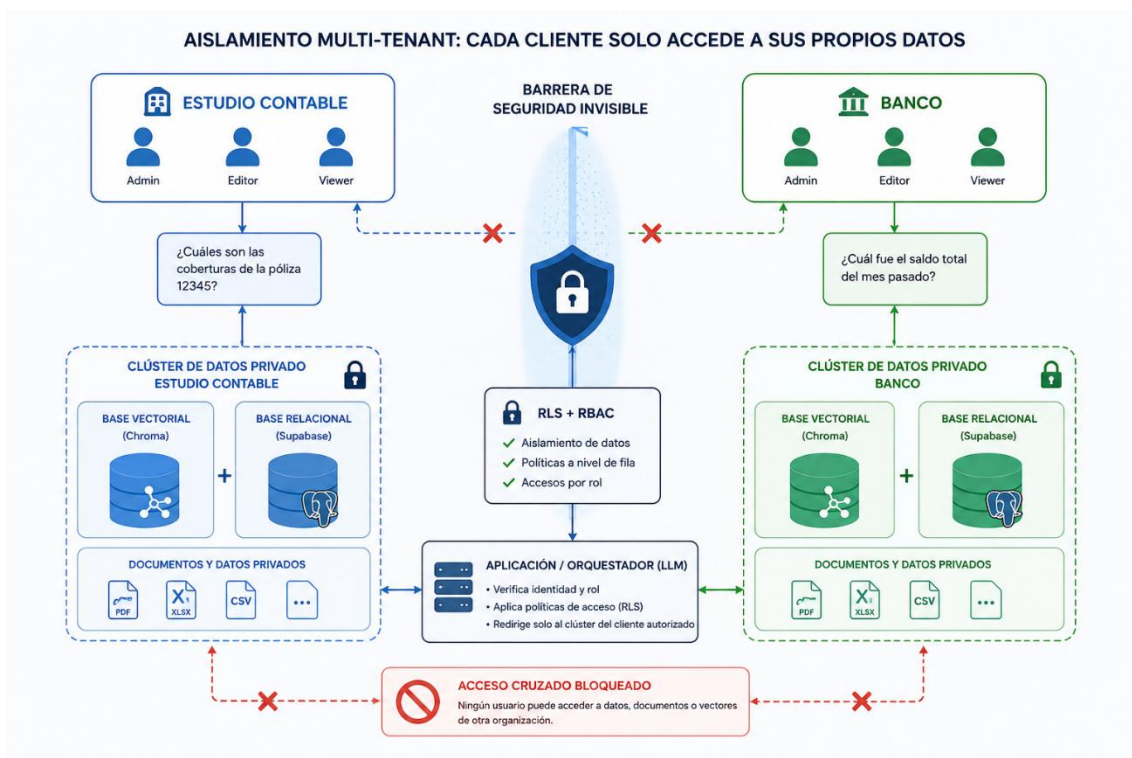
Para asegurar que esta plataforma sea un producto B2B verdaderamente viable y seguro en el mundo real, tuvimos que construir un entorno donde cada empresa opera

- en un espacio totalmente hermético
- garantizando así la privacidad de sus datos
- un funcionamiento constante sin costes desorbitados de servidores.

A continuación, veremos las dos claves de este logro: cómo protegemos la información corporativa y cómo estructuramos nuestra red de forma inteligente.

H2 Seguridad y Aislamiento de la Información Corporativa

El primer pilar es la seguridad. Un cliente jamás debe tener acceso a los documentos o datos de otro cliente. **A esto lo llamamos un entorno aislado.** Además de proteger los datos entre diferentes empresas, **nuestro sistema controla estrictamente quién entra y qué puede hacer cada empleado dentro de su propia organización.** Si un empleado no tiene los permisos suficientes, la plataforma simplemente se adapta y oculta botones u opciones críticas, asegurando que un usuario básico pueda consultar la Inteligencia Artificial, pero nunca borrar información estratégica.



H3 **Gestión de Identidad en el Edge (Clerk):**

Hemos integrado Clerk como solución de autenticación moderna compatible con arquitecturas Edge, evaluando el acceso en el middleware de Next.js antes de renderizar la página y ofreciendo flujos de acceso avanzados (Email y credenciales clásicas).

- **¿Qué es el Edge?** Significa que el código de seguridad no se ejecuta en un único servidor lejano, sino en una red global de servidores ultra-rápidos de Vercel distribuidos por todo el mundo, operando en el nodo más cercano al usuario.

- **¿Qué es el Middleware?** Es un "guardia de seguridad" a la entrada de la aplicación. Inspecciona al usuario antes de que la página web empiece siquiera a cargarse o renderizarse en el navegador.
- **¿Qué hace Clerk aquí?** Es la herramienta que gestiona los usuarios y las empresas. Emite un pase digital (Token JWT) que el Middleware lee al instante para comprobar quién eres y a qué empresa perteneces.

¿Por qué es genial esta solución? (El valor técnico)

1. **Seguridad Total:** Bloquea a los intrusos en la periferia de la red, antes de que puedan tocar vuestros datos o consumir recursos del sistema.
2. **Velocidad Absoluta:** Al verificar la identidad en el servidor más cercano al usuario y sin consultar bases de datos lentas en cada clic, la aplicación responde de manera inmediata.
3. **Control Multi-Empresa:** Permite estructurar de forma limpia los accesos para que cada cliente (como el Estudio Contable o el Banco) se mantenga en su entorno privado.

H3 ****Control de Acceso Basado en Roles (RBAC) y Sincronización:****

Clerk emite un Token JWT con un `publicMetadata` que contiene el `orgId` (identificador de la empresa) y el `role` (`admin`, `editor`, `viewer`). Un hook especializado en React (`AuthContext.tsx`) captura este JWT en el inicio de sesión y sincroniza el perfil mediante una operación `upsert` en la tabla relacional `user\_profiles` de Supabase.

- **¿Qué es RBAC?** Significa "Control de Acceso Basado en Roles". Es el sistema que decide qué puede hacer cada empleado según su rango: **Admin** (control total), **Editor** (modifica datos) o **Viewer** (solo lee).
- **El "Pase VIP" (Token JWT con publicMetadata):** Cuando el usuario inicia sesión, **Clerk** le entrega una tarjeta digital de identificación (el Token JWT). Dentro de esta tarjeta viene grabado a fuego un chip invisible (publicMetadata) que dice dos cosas esenciales: **a qué empresa perteneces** (orgId) y **qué puesto tienes** (role).
- **El Receptor (AuthContext.tsx):** Es un componente espía en la interfaz de usuario (React). En cuanto el usuario entra, este componente "caza" la tarjeta digital (el JWT) para saber al instante qué botones debe mostrarle u ocultarle en la pantalla.
- **La Sincronización (upsert en Supabase):** Para que la base de datos sepa también quién ha entrado, el sistema hace un upsert (un comando inteligente que significa: *"Si el usuario ya existe en la base de datos, actualiza sus datos; si es nuevo, regístralo"*). Así, la tabla de perfiles en **Supabase** se mantiene idéntica a la de Clerk en tiempo real.

¿Por qué es genial esta solución? (El valor técnico)

1. **Eficiencia Máxima:** Al guardar el rol y la empresa dentro de la tarjeta digital (JWT), el frontend no tiene que preguntar a la base de datos qué permisos tiene el usuario cada vez que hace un clic. El pase ya lleva toda la información encima.

2. **Consistencia de Datos:** La operación upsert garantiza que la base de datos de la aplicación y el sistema de autenticación jamás se descoordinen, evitando errores de permisos.

H3 Aislamiento Físico de Datos (Row Level Security - RLS)

Para garantizar el Multi-Tenancy a nivel de infraestructura profunda, implementamos políticas de seguridad a nivel de fila (RLS) nativas de PostgreSQL en Supabase. Al inyectar dinámicamente el `orgId` en cada consulta, la base de datos rechaza a nivel criptográfico cualquier lectura o escritura de registros que no pertenezcan a la organización del usuario, neutralizando el riesgo de "Cross-Tenant Data Leakage".

- **¿Qué es el Multi-Tenancy?** Es el modelo de software donde una sola aplicación da servicio a múltiples empresas clientes (tenants), pero cada una debe vivir en su propio espacio estanco.
- **¿Qué es el RLS (Row Level Security)?** Tradicionalmente, si tienes acceso a una tabla de la base de datos, ves *todas* las filas. RLS es una funcionalidad de PostgreSQL (la base de datos que usa Supabase) que actúa como un filtro invisible e infranqueable directamente **en cada fila de la base de datos**.
- **El mecanismo de bloqueo:** Cada vez que el código hace una consulta (ej. *"busca este documento"*), el sistema inyecta automáticamente el código identificador de la empresa (orgId). La base de datos mira fila por fila y dice: *"¿Esta fila coincide con el orgId del usuario? Si sí, se la muestro; si no, la oculto como si jamás hubiera existido"*.

¿Por qué es genial esta solución? (El valor técnico)

1. **Seguridad Blindada (A nivel criptográfico):** El filtrado no se hace mediante código tradicional en el backend (donde un programador podría olvidar añadir un WHERE tenant_id = X por error). Lo gestiona el propio motor profundo de la base de datos. Si un atacante intentara trucar la URL o la interfaz para ver datos de otra empresa, la base de datos rechazaría la petición de raíz.
2. **Neutraliza el Cross-Tenant Data Leakage:** Elimina al 100% el riesgo de filtración de datos entre empresas clientes. Un Banco jamás podrá ver, ni por error, un solo registro del Estudio Contable.

H3 **Prevención del Cross-Tenant Data Leakage en la Capa Visual:**

La arquitectura de roles condiciona completamente el DOM de Next.js. A los perfiles con rol `viewer`, el servidor les re-renderiza la interfaz bloqueando dinámicamente el acceso a funcionalidades de manipulación de archivos y diccionarios corporativos, creando una degradación elegante del sistema.

- **¿Qué es el DOM de Next.js?** El DOM (Document Object Model) es, básicamente, la estructura de la página web que ve y toca el usuario (los botones, los menús, los formularios).
- **El mecanismo de bloqueo visual:** Cuando un usuario entra a la plataforma, el servidor de Next.js comprueba su rol antes de enviarle la página. Si su rol es viewer (un observador básico), el servidor **reconfigura la interfaz en tiempo real**.

- **¿Qué es la Degradación Elegante?** En lugar de lanzar un error rudo de "Acceso Denegado" o romper la página, la aplicación se adapta "elegantemente". Al usuario con rol viewer se le ocultan o deshabilitan por completo los botones peligrosos (como "Subir archivo", "Borrar documento" o "Modificar diccionario corporativo"). La app sigue funcionando perfectamente, pero limitada a sus permisos.

¿Por qué es genial esta solución? (El valor técnico)

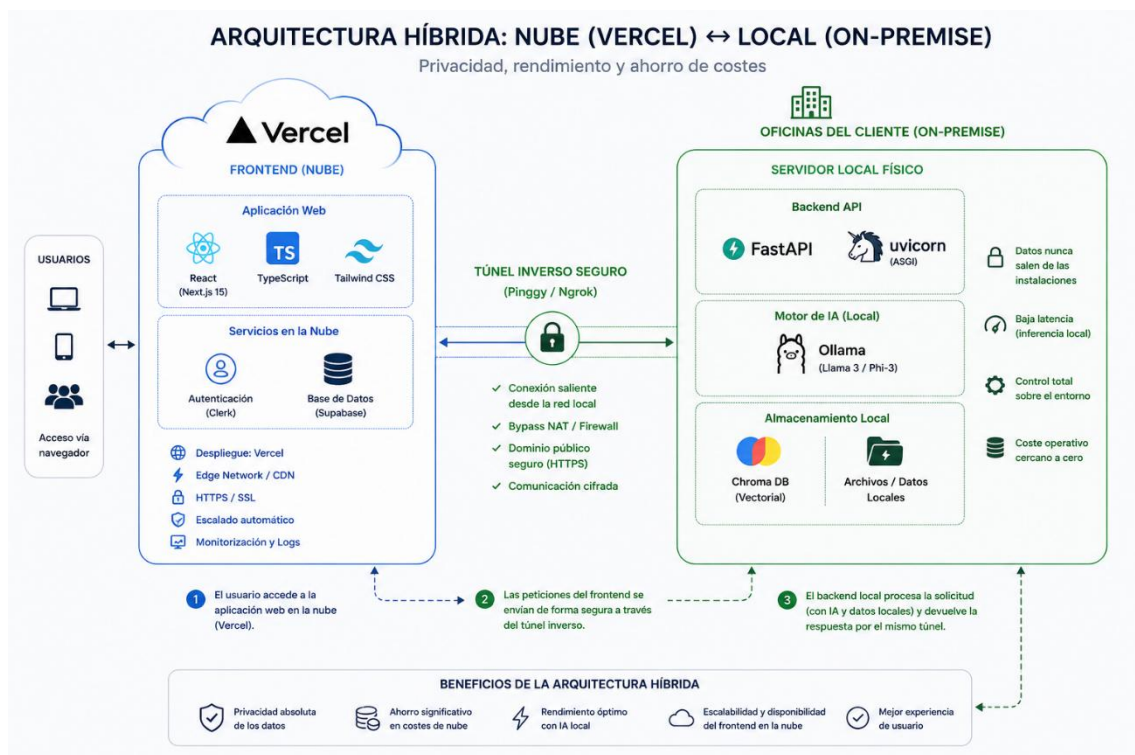
1. **Doble Capa de Seguridad:** No basta con bloquear los datos en la base de datos profunda (con el RLS). Al bloquear también los elementos en la Capa Visual, evitáis que un usuario intente siquiera pulsar un botón para el que no está autorizado.
2. **Excelente Experiencia de Usuario (UX):** Evita la frustración. El empleado solo ve las herramientas que realmente puede usar, manteniendo una interfaz limpia, intuitiva y segura.

H2 Despliegue Híbrido: Privacidad Absoluta y Ahorro de Costes

El segundo gran reto operativo fue la conectividad. **Alquilar potentes ordenadores en la nube** para alojar y hacer funcionar nuestro cerebro de Inteligencia Artificial hubiera **disparado los costes** a cientos de euros mensuales, haciendo el proyecto inviable. Por ello, diseñamos una **arquitectura "híbrida"**:

- la interfaz gráfica y la aplicación web que usan los clientes están alojadas en la nube
- el núcleo de razonamiento de la Inteligencia Artificial se ejecuta directamente en los servidores locales físicos de la empresa.

Gracias a esto, no pagamos por servidores en la nube, las respuestas son instantáneas y la empresa mantiene el control 100% sobre sus propios datos sin depender de terceros.



H3 ****Túnel Reverso para Bypassing NAT:****

Al ejecutar el frontend en Vercel (Cloud) y el backend en FastAPI local (On-Premise), la red Serverless de Vercel arrojaba errores 503/404 por culpa de los cortafuegos y el NAT local. Implementamos un túnel reverso persistente y seguro mediante ****Pinggy / Ngrok****, inyectando un dominio estático (`PYTHON_BACKEND_URL``) en las variables de entorno de Vercel.

H3 ****Resolución del Hydration Mismatch en Next.js:****

A nivel de interfaz, el nuevo ***App Router*** de Next.js generó conflictos de hidratación porque el servidor (Vercel) pre-renderizaba la barra lateral sin la sesión autenticada, mientras el cliente (navegador) tenía la cookie de Clerk activa. Unificamos el manejo de estado asíncrono en el ``RootLayout`` para lograr una hidratación síncrona sin colapsos visuales.

- **¿Qué es la Hidratación?** En Next.js, para que la web cargue súper rápido, el servidor (Vercel) dibuja primero una versión estática de la página (HTML plano) y se la envía al navegador. Justo después, el navegador descarga el código JavaScript de **React** y lo "acopla" sobre ese dibujo estático para que los botones tengan vida. A ese proceso de darle vida al código se le llama **hidratación**.
- **La regla de oro:** Para que React no se rompa, el dibujo que hace el servidor y el dibujo que arranca en el navegador **tienen que ser exactamente idénticos**.
- **El conflicto (El Mismatch):** Al usar el nuevo *App Router* de Next.js, ocurrió una descoordinación:
 1. **El servidor** dibujaba la barra lateral asumiendo que el usuario *no* estaba logueado porque no tenía acceso inmediato a los datos en tiempo real.
 2. **El navegador** del usuario, al recibir la página, detectaba que la cookie de inicio de sesión de **Clerk** sí estaba activa y dibujaba la barra lateral con las opciones del usuario logueado.
- **El resultado:** Al no coincidir las dos versiones, React lanzaba un error crítico en la consola (*Hydration Mismatch*), provocando parpadeos molestos o "colapsos visuales" en la pantalla.

¿Cómo lo solucionasteis? (Unificación en el RootLayout)

En lugar de dejar que cada componente de la barra lateral intentara adivinar de forma asíncrona si el usuario estaba dentro o fuera, movisteis y centralizasteis toda la lógica del estado de autenticación en la raíz de la aplicación: el **RootLayout**.

Al envolver toda la estructura en un único punto de control síncrono, garantizasteis que tanto el servidor como el cliente se pusieran de acuerdo sobre el estado de la sesión antes de pintar cualquier píxel, logrando una carga limpia, fluida y sin errores de consola.

¿Por qué es genial esta solución? (El valor técnico)

Demuestra ante el tribunal que domináis el ciclo de vida del renderizado de **Next.js** y que sabéis resolver problemas avanzados de sincronización entre el servidor (*Server-Side Rendering*) y la interfaz del cliente.

H3 ****Configuración de Red y Timeouts:****

Debido a que el LLM genera respuestas en streaming (SSE) y a veces tiene alta latencia de inferencia en hardware local, el ruteo asíncrono requirió un rediseño manual en el archivo `vercel.json` para extender la tolerancia de red de larga duración en la capa de Vercel y evitar cortes de conexión abruptos.

- **¿Qué es el Streaming (SSE)?** Los modelos de lenguaje (LLM) no devuelven la respuesta de golpe como una base de datos tradicional. En su lugar, usan **Server-Sent Events (SSE)**, que es la tecnología que permite que la IA vaya respondiendo y "escribiendo" la respuesta palabra por palabra en vuestra pantalla en tiempo real.
- **El problema de la latencia local:** Como vuestro backend de IA corre en un servidor local (On-Premise) y no en un superordenador en la nube, el "cerebro" tarda un poco más en pensar (alta latencia de inferencia) antes de empezar a escupir las palabras.
- **El corte abrupto de Vercel (Timeout):** Las plataformas en la nube como **Vercel** tienen una regla estricta por defecto: si un servidor tarda más de unos pocos segundos (normalmente 10-15 segundos) en responder, Vercel asume que el sistema se ha colgado y **corta la conexión de golpe** (lanzando un error de red). Esto hacía que las respuestas largas de vuestra IA se quedaran a medias.

¿Cómo lo solucionasteis? (Rediseño en vercel.json)

Para evitar que la nube colgara el teléfono a vuestro servidor local mientras este seguía procesando el texto, modificasteis manualmente el archivo de configuración **vercel.json** de vuestro proyecto.

En este archivo añadisteis una directiva para **extender los límites de tiempo de espera (timeouts)** de las funciones *Serverless*. Le dijisteis a Vercel algo como: *"Oye, mantén la conexión abierta durante más tiempo del habitual porque estamos transmitiendo datos de IA de larga duración desde un entorno local"*.

¿Por qué es genial esta solución? (El valor técnico)

1. **Estabilidad del Sistema:** Asegura que los usuarios puedan recibir respuestas completas, complejas y detalladas de la IA sin sufrir cortes inesperados o pantallas de error a mitad del chat.
2. **Optimización Cloud-to-Local:** Demuestra que sabéis configurar infraestructura en la nube real para adaptarla a las limitaciones físicas del hardware local, logrando que dos mundos distintos se entiendan a la perfección.

Speech

Para que nuestra plataforma sea un producto B2B verdaderamente viable, tuvimos que construir un entorno donde cada empresa opera

- en un espacio totalmente hermético
- garantizando así la privacidad de sus datos
- un funcionamiento constante sin costes desorbitados de servidores.

A continuación, veremos las dos claves de este logro:

- cómo protegemos la información corporativa
- cómo estructuramos nuestra red de forma inteligente.

H2 Seguridad y Aislamiento de la Información Corporativa

El primer pilar es la seguridad. Un cliente jamás debe tener acceso a los documentos o datos de otro; a esto lo llamamos un entorno aislado. En línea con lo anterior, también manejamos quién entra y quién sale, así como qué puede hacer cada empleado, **gestión que realizamos a través de Clerk**. Si un empleado no tiene permisos para editar documentos, el sistema no se lo permite.

Como se ve en la imagen, tenemos ambos clientes con sus respectivos empleados, pero ninguno puede acceder a la información del otro. Esto se debe a que toda la administración de la base de datos **en Supabase cuenta con políticas de Row Level Security (RLS)**, las cuales se encargan de filtrar, mostrar u ocultar la información dependiendo estrictamente del rol y del entorno del usuario.

Despliegue Híbrido: Privacidad Absoluta y Ahorro de Costes

El segundo gran pilar es la conectividad. **Alquilar potencia de cómputo o servidores** en la nube para alojar y **ejecutar la aplicación web y el modelo de IA** hubiese disparado los costes. Por eso, implementamos una arquitectura híbrida:

- la interfaz gráfica y la aplicación web que usan los clientes están alojadas en la nube, **mientras que**
- el núcleo de razonamiento de la Inteligencia Artificial se ejecuta directamente en los servidores locales físicos de la empresa.

Como se muestra en el diagrama, a la izquierda tenemos nuestro hosting con las herramientas de la interfaz. **Por otro lado, en el nodo local, contamos con** las herramientas que usamos para hacer funcionar **y dar contexto a nuestro modelo**, donde:

- **FastAPI** se encarga de hacer de puente entre las consultas del usuario y la IA.
- **Uvicorn** hace que FastAPI se mantenga activo y escuchando en tiempo real.
- **ChromaDB actúa como nuestra** base de datos vectorial para el contexto de la IA.

Y en el medio, el túnel de **Ngrok** permite que la página web (alojada en la nube de Vercel) pueda conectarse directamente con nuestra IA local de forma privada, sin necesidad de configurar redes complejas ni pagar por IPs públicas."